

Class 6: R Functions

David Tin (PID: A17785446)

Table of contents

Background	1
A first function	1
A <code>generate_dna()</code> function	3
Write a <code>generate_protein()</code> function	6
Generate random protein sequences of length 6 to 13	6
Are Our peptides “unique in nature”?	8
Connecting our findings to immunology	9

Background

All functions in R have at least 3 things:

- a **name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either “**required**” or “**optional**”. The latter have fall-back default values that will be used if the user does not specify them.

```
add <- function(x, y=0) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [1 pt]

```
add <- function(x, y=0) {  
  sum(x, y)  
}
```

```
add(4,7)
```

```
[1] 11
```

```
add( c(4,7,10) )
```

```
[1] 21
```

Q1c. To do on your own... create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)`

```
add <- function(x, y=0, ...) {  
  sum(x, y, ...)  
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function

```
add <- function(x, y=0) {
  sum(x, y)
  cat("Is it breaktime yet?\n")
}
```

A generate_dna() function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=6, replace = T)
```

```
[1] 2 4 3 3 2 5
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A", "G", "T", "C"), size=10, replace=T)
```

```
[1] "T" "A" "T" "G" "T" "C" "G" "T" "T" "G"
```

Q2a: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len=10){
  sample(x=c("A", "G", "T", "C"), size=len, replace=T)
}
```

```
generate_dna(len=10)
```

```
[1] "G" "C" "C" "A" "C" "T" "T" "A" "T" "G"
```

Q2b: Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

My code

```
generate_dna <- function(len=10, y=T){
  if(y==T){paste(sample(x=c("A", "G", "T", "C"), size=len,
                           replace=T),collapse="")}
  }else
  return((sample(x=c("A", "G", "T", "C"), size=len, replace=T)))
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna(10,T)
```

```
[1] "TCCGGGCCAT"
```

Example in class

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A","C","G","T"), size=len, replace=T)
  #cat("Hello....")

  if(single.element) {
    #cat("is it me you are looking for...")
    ans <- paste(ans,collapse="")
  }
  return(ans)
}
```

```
generate_dna(single.element = F)
```

```
[1] "G" "G" "C" "A" "C" "C" "T" "A" "A" "C"
```

Q2c: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9
CGAAGGCTG
```

```
cat("hello\n there")
```

```
hello
there
```

My code:

```
generate_dna <- function(len=10, y=T){  
  ##Format as FASTA with an ID line  
  cat(">len",len,"\n",sep="")  
  ##Generate nucleotide sequence as a continuous string  
  if(y==T){cat(sample(x=c("A", "G", "T", "C"), size=len, replace=T),sep="")  
  }else  
  ##Generate sequence as multi-element vector  
  return((sample(x=c("A", "G", "T", "C"), size=len, replace=T)))  
}
```

```
generate_dna(15,T)
```

```
>len15  
TCGTTTCCGAACGTG
```

Example code in class:

```
generate_dna <- function(len=10, single.element=TRUE) {  
  ans <- sample(x=c("A","C","G","T"), size=len, replace=T)  
  
  if(single.element) {  
    ans <- paste(ans,collapse="")  
  }  
  
  ##Format as FASTA with an ID line  
  cat(paste(">len",len,"\n",sep=""))  
  cat(ans)  
  cat("\n")  
  ##  
}
```

```
generate_dna(42,T)
```

```
>len42  
ACCCCGTGTAAGACAACAGGTGGGCCTATATTATAGTATATG
```

Write a generate_protein() function

Q3: Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
generate_protein <- function(len=10){  
  ## Assigning a variable that is responsible  
  ## for generating a random protein sequence  
  pro <- sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L",  
                    "K", "M", "F", "P", "S", "T", "W", "Y", "V"),  
               size=len, replace=T)  
  ## Generating the protein sequence as a continuous string  
  cat(pro, sep="")  
}
```

```
generate_protein(6)
```

DYNEHT

Generate random protein sequences of length 6 to 13

```
generate_protein <- function(len){  
  ## Creating loop that repeats the function from lengths 6:13  
  for(len in 6:13){  
    ## FASTA format  
    cat(paste(">id.",len,"\n",sep=""))  
    ## Protein sequence generation  
    cat(sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L",  
                  "K", "M", "F", "P", "S", "T", "W", "Y", "V"),  
            size=len, replace=T),"\n",sep="")  
  }  
}
```

```
generate_protein()
```

```
>id.6  
HQMEAR
```

```

>id.7
YVRWPTM
>id.8
RQNIGQTI
>id.9
CQKIAKHHA
>id.10
AYSFSDCPTG
>id.11
FNYIPAIFTKC
>id.12
PKKQQQVARGNP
>id.13
NESTCWHEACHYV

```

Example code in class:

```

generate_protein <- function(len){
  ## Assigning a variable that is responsible for
  ## generating a random protein sequence
  pro <- sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I",
                    "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V"),
               size=len, replace=T)
  ## Generating the protein sequence as a continuous string
  cat(pro, sep="")
}

```

```

for(l in 6:13){
  cat(">",l, "\n",sep="")
  cat(generate_protein(l), "\n")
}

```

```

>6
HDDGQC
>7
IGPSCWN
>8
CFPRYNYP
>9
YLEDPGPDN
>10
VKPPAEIRYV

```

```

>11
RNRIHANMKKP
>12
SAEQCLARWKPG
>13
AHEETDCSWRTRM

```

Are Our peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

length	Best hit % identity	Best hit % coverage	Unique (Y/N)
6	100%	100%	N
7	100%	100%	N
8	100%	100%	N
9	100%	100%	N
10	100%	80%	Y
11	81.82%	91%	Y
12	81.82%	92%	Y
13	100%	69%	Y

Q5a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

At around a sequence length of 10, the randomly generated peptides started to look “unique in nature”.

Q5b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20ⁿ for a peptide of length n) and to the size of the known protein universe.

Short random chains can be found randomly, as their short length allows for greater chances of happening purely by chance. The longer the peptide, the less likely it is that a match is by chance. Because there are 20 amino acids, each amino acid you add to a peptide, it greatly increases the uniqueness of the sequence. A sequence of 3 only has 8000 possible sequences, whereas a sequence of 10 has 1.024e13 possible sequences.

Connecting our findings to immunology

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on the results I found in Q5, I believe that 10 amino acids in length may be an optimal minimum length. Very short peptides in the immune system may cause it to recognize sequences that it does not intend to, and end up attacking essential proteins in the body. Having sequences that are unique are essential to T-cell receptor recognition to distinguish pathogens from proteins in the body.